

Exception Handling

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor (CSE), PIT
Parul University

1. Exception Handling.....	1
2. Java Exception Hierarchy.....	1
3. Difference Between Errors and Exceptions	3
4. Summary of Key Concepts.....	4
5. References.....	4

8.1.1 Exception Handling

◆ What is an Exception in Java?

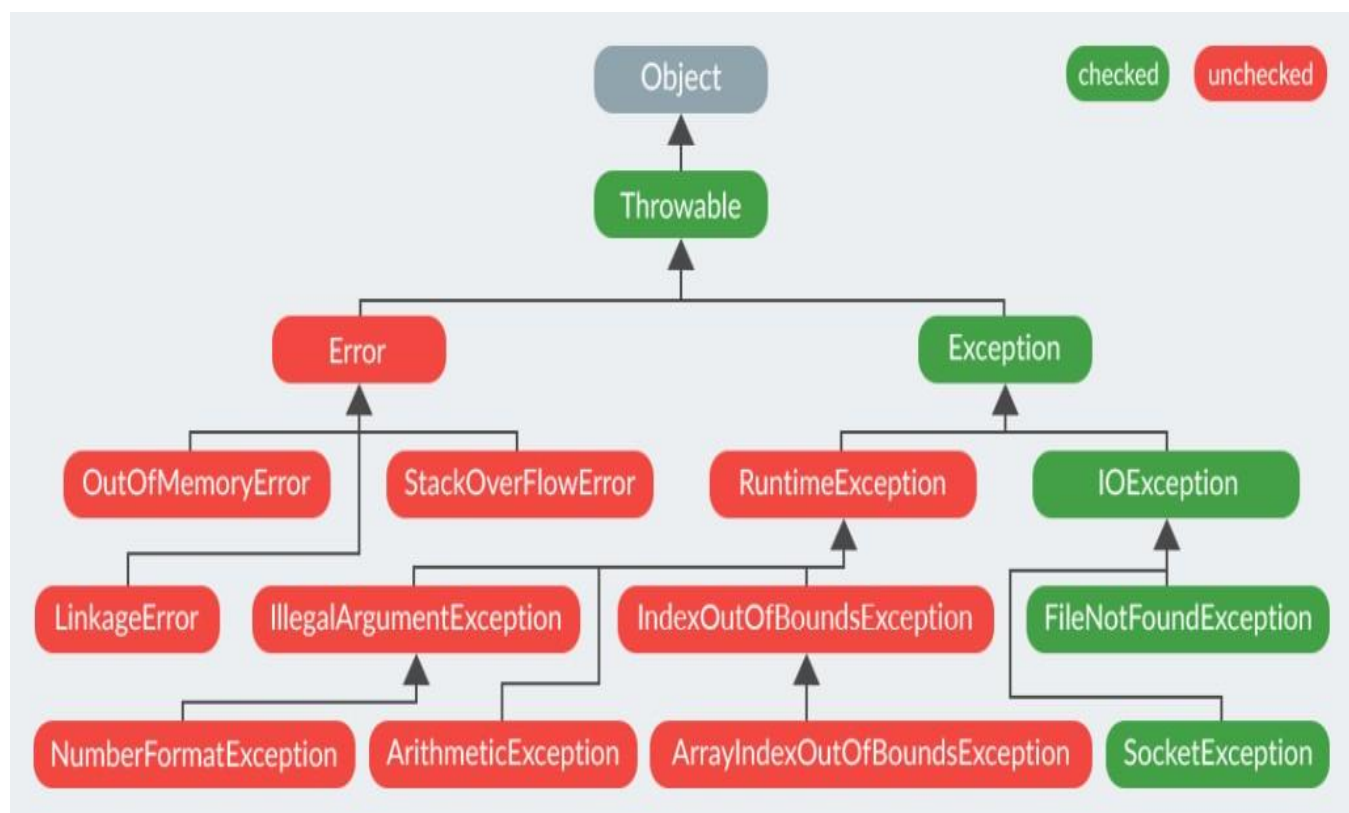
In Java, an **exception** is an **event** that disrupts the normal flow of a program's instructions. It occurs **during runtime** and indicates that something went wrong (e.g., dividing by zero, accessing an array out of bounds, etc.).

◆ Why Use Exception Handling?

- To **maintain the normal flow** of the application.
- To provide **informative error messages** to the user.
- To **handle errors gracefully** rather than terminating the program abruptly.
- To **separate error-handling code** from normal code, increasing readability.

8.1.2 Java Exception Hierarchy

In Java, exceptions are part of the `java.lang.Throwable` class hierarchy. The hierarchy is structured to differentiate between errors and exceptions, allowing developers to handle them appropriately. Here's an overview of the hierarchy:



Exception Hierarchy

Source: <https://codegym.cc/images/article/910d576f-391b-4441-8e46-11e7e7d9d71a/1024.jpeg>

1. Throwable

The root class for all exceptions and errors in Java.

It has two main subclasses:

- **Error**
- **Exception**

2. Error

Represents serious problems that applications should not try to catch.

Typically related to the Java Virtual Machine (JVM) and system-level issues.

Examples:

- **OutOfMemoryError**
- **StackOverflowError**
- **VirtualMachineError**

3. Exception

Represents conditions that an application might want to catch and handle.

Subdivided into:

- **Checked Exceptions**
- **Unchecked Exceptions (Runtime Exceptions)**

1. Checked Exceptions

- Checked at **compile time**.
- Must be handled using **try-catch** or declared with **throws**.

Examples:

- **IOException**
- **SQLException**
- **FileNotFoundException**

```
public void readFile() throws IOException {  
    FileReader file = new FileReader("file.txt");  
}
```

2. Unchecked Exceptions

- Checked at **runtime**.
- **Not mandatory** to handle.
- Subclasses of RuntimeException.

Examples:

- NullPointerException
- ArithmeticException
- IndexOutOfBoundsException

```
int result = 10 / 0; // ArithmeticException
```

8.1.3 Difference Between Errors and Exceptions

Feature	Errors	Exceptions
Definition	Serious issues not handled in code	Issues that can be handled by the program
Occurs When?	Mostly during runtime (e.g., JVM failure)	Runtime or compile time
Examples	OutOfMemoryError, StackOverflowError	IOException, NullPointerException
Recoverable?	Usually not recoverable	Often recoverable with proper handling
Subclass of?	java.lang.Error	java.lang.Exception

◆ Benefits of Exception Handling:

- Makes the program more robust and fault-tolerant.
- Enhances user experience by handling errors properly.
- Ensures code reliability in real-world systems.

📌 Example:

```
public class Example {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // Throws ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }
        System.out.println("Program continues...");
    }
}
```

➤ Summary of Key Concepts:

- **Exception:** An abnormal condition disrupting normal program flow.
- **Errors vs. Exceptions:**
 - Errors are critical system issues (e.g., `OutOfMemoryError`) and generally not recoverable.
 - Exceptions are conditions that a program can handle and recover from (e.g., `IOException`, `ArithmeticException`).
- **Java Exception Hierarchy:**
 - All exceptions and errors derive from `Throwable`.
 - `Throwable` splits into:
 - Error (e.g., system crashes),
 - Exception (program-level problems).
- **Types of Exceptions:**
 - **Checked Exceptions** (e.g., `IOException`): Must be handled at compile time.
 - **Unchecked Exceptions** (e.g., `NullPointerException`): Occur at runtime, optional to handle.

➤ References:

- Oracle Java Documentation □ Exception Handling
 <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- GeeksforGeeks □ Exceptions in Java
 <https://www.geeksforgeeks.org/exceptions-in-java/>
- JavaTpoint □ Exception Handling in Java
 <https://www.javatpoint.com/exception-handling-in-java>
- Baeldung □ Guide to Exceptions in Java
 <https://www.baeldung.com/java-exceptions>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>